Output

Today's Topics

- Output approaches
- Drawing
- Rasterization
- Declarative programming

Three Output Approaches

- Objects
  - Graphical objects arranged in a tree with automatic redraw
  - Example: Label object, Line object
  - Also called: views, interactors, widgets, controls, retained graphics

HTML

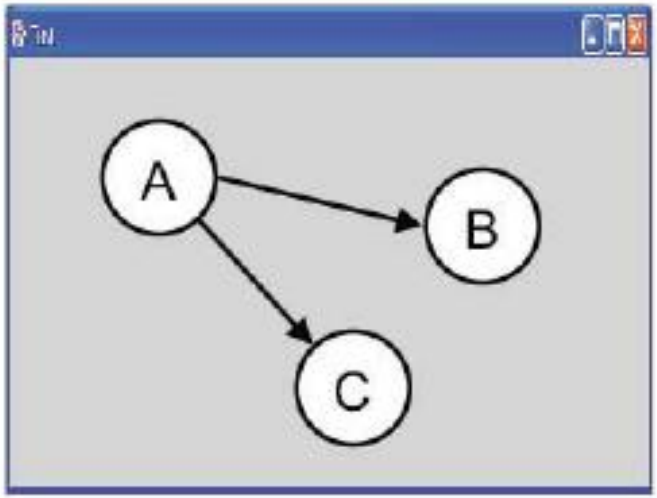
- Strokes
  - High-level drawing primitives: lines, shapes, curves, text
  - Example: drawText() method, drawLine() method
  - Also called: vector graphics, structured graphics

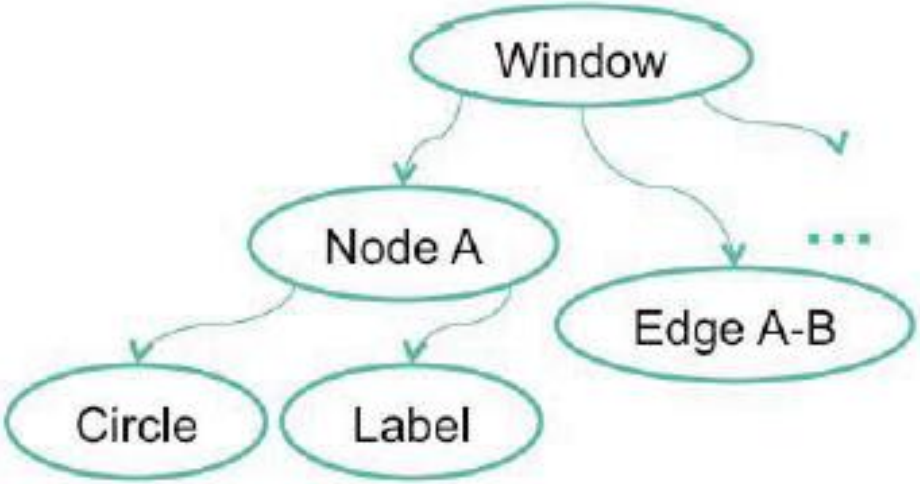
PDF

- Pixels
  - 2D array of pixels
  - Also called: raster, image, bitmap

LCD panel

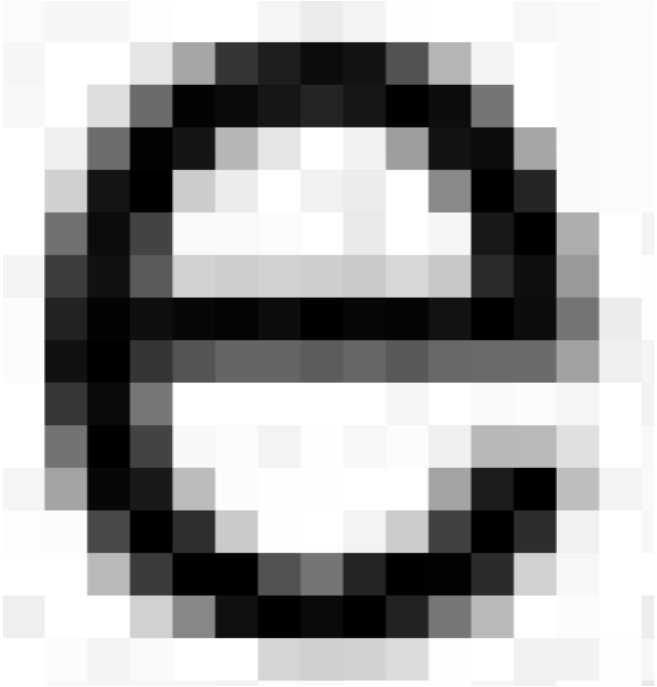
Example: Designing a Graph View



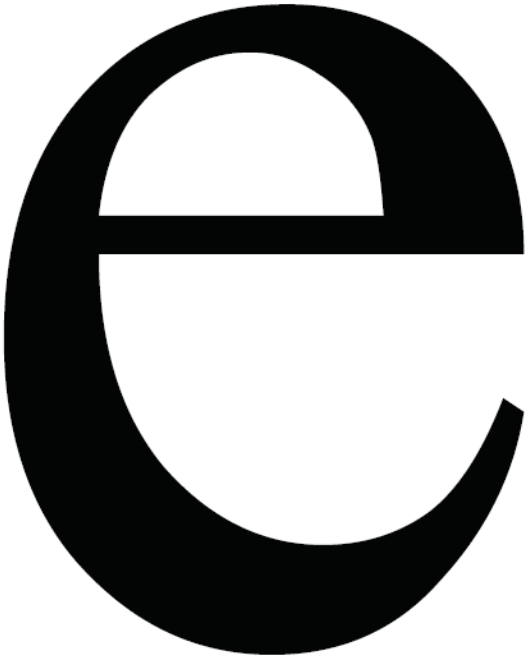


- Component model
  - Each node and edge is a component
  - A node might have two subcomponents: circle and label
- Stroke model
  - Graph view draws lines, circles and text
- Pixel model
  - Graph view has pixel images of the nodes

Hybrid approaches for the graph view are certainly possible, in which some parts of the output use one approach, and others use another approach



When you zoom into a part of image in Acrobat Reader



When you zoom into a character in Acrobat Reader

Layout: Objects remember where they were put, and draw themselves there. They also support automatic layout

With strokes or pixls, you have to figure out (at drawing time) where each piece goes, and put it there.

Input: Objects participate in event dispatch and propagation, and the system automatically does hit-testing. If a graph node is an object, then it can receive its own click and drag events

If you stroked the node instead, then you have to write code to determine which node was clicked or dragged.

Redraw: components redraw themselves automatically when they have to. redraw algorithm is efficient: it only redraws components whose extents intersect the damaged region

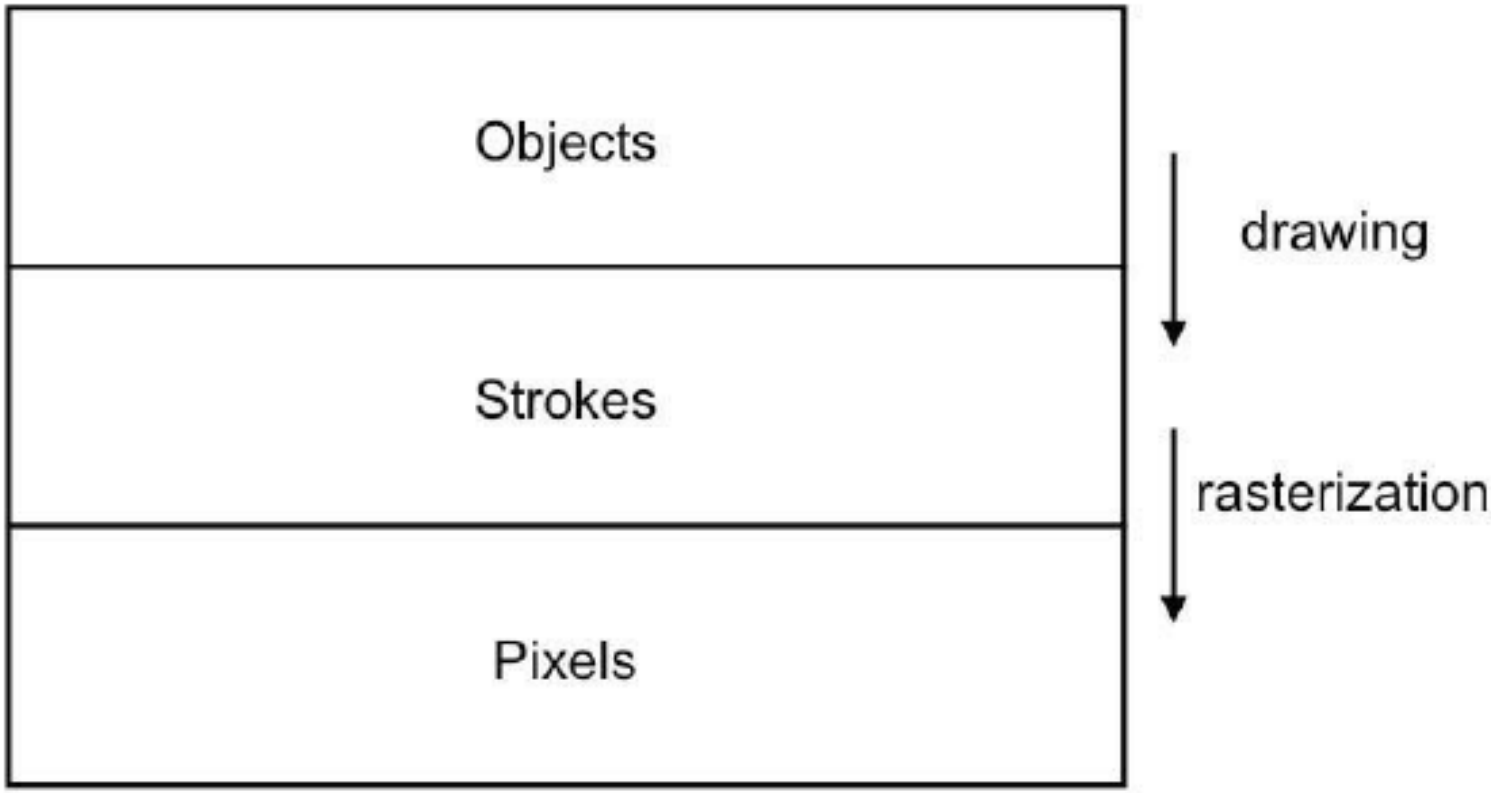
The stroke or pixel model would have to do this test by hand. In practice, most stroked objects don’t bother, simply redrawing everything whenever some part of the view needs to be redrawn.

Drawing order: It’s easy for a parent to draw before (underneath) or after (on top of) all of its children. But it’s not easy to interleave parent drawing with child drawing. if you’re using a hybrid model, with some parts of your view represented as components and others as strokes, then the components and strokes generally fall in two separate layers, and you can’t have any complicated layering relationships between strokes and components.

Heavyweight objects: Objects may be big: the view tree is overloaded not just with drawing functions but also with event dispatch, automatic redraw, and automatic layout, so the properties and state used by those processes further bulks up the class. Views derived from large amounts of data – say, a 100,000-node graph – generally can’t use an object for every individual data item

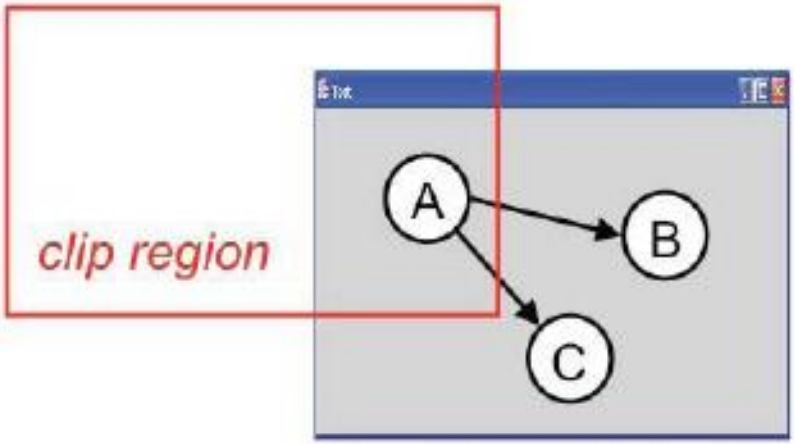
Device dependence: The stroke approach is largely device independent. In fact, it’s useful not just for displaying to screens, but also to printers, which have different resolutions.
The pixel approach, on the other hand, is device dependent. A directly mapped pixel image won’t look the same on a screen with a different resolution.

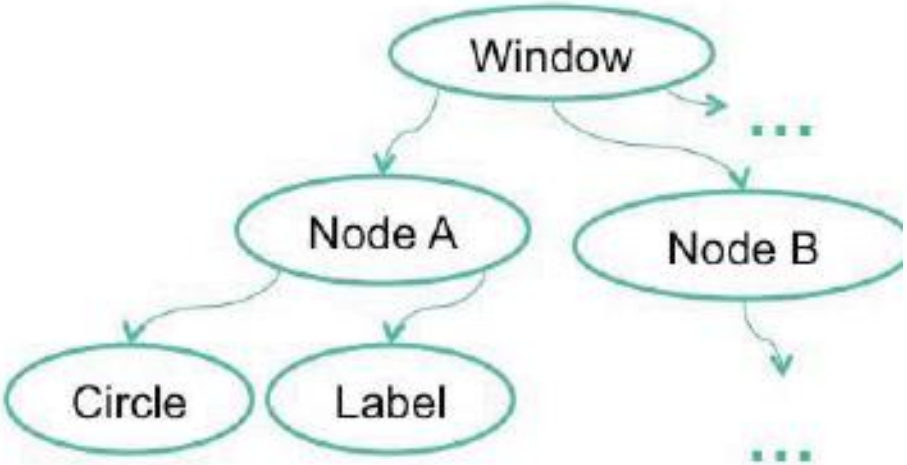
How Output Approaches Interact



Drawing a View Tree

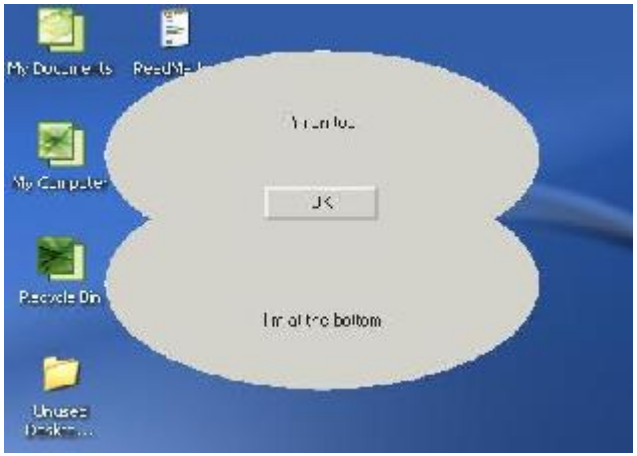
- Drawing goes top down
  - Draw self (using strokes or pixels)
  - For each child component,
    - If child intersects clipping region then
      - intersect clipping region with child's bounding box
      - recursively draw child with clip region set to the intersection



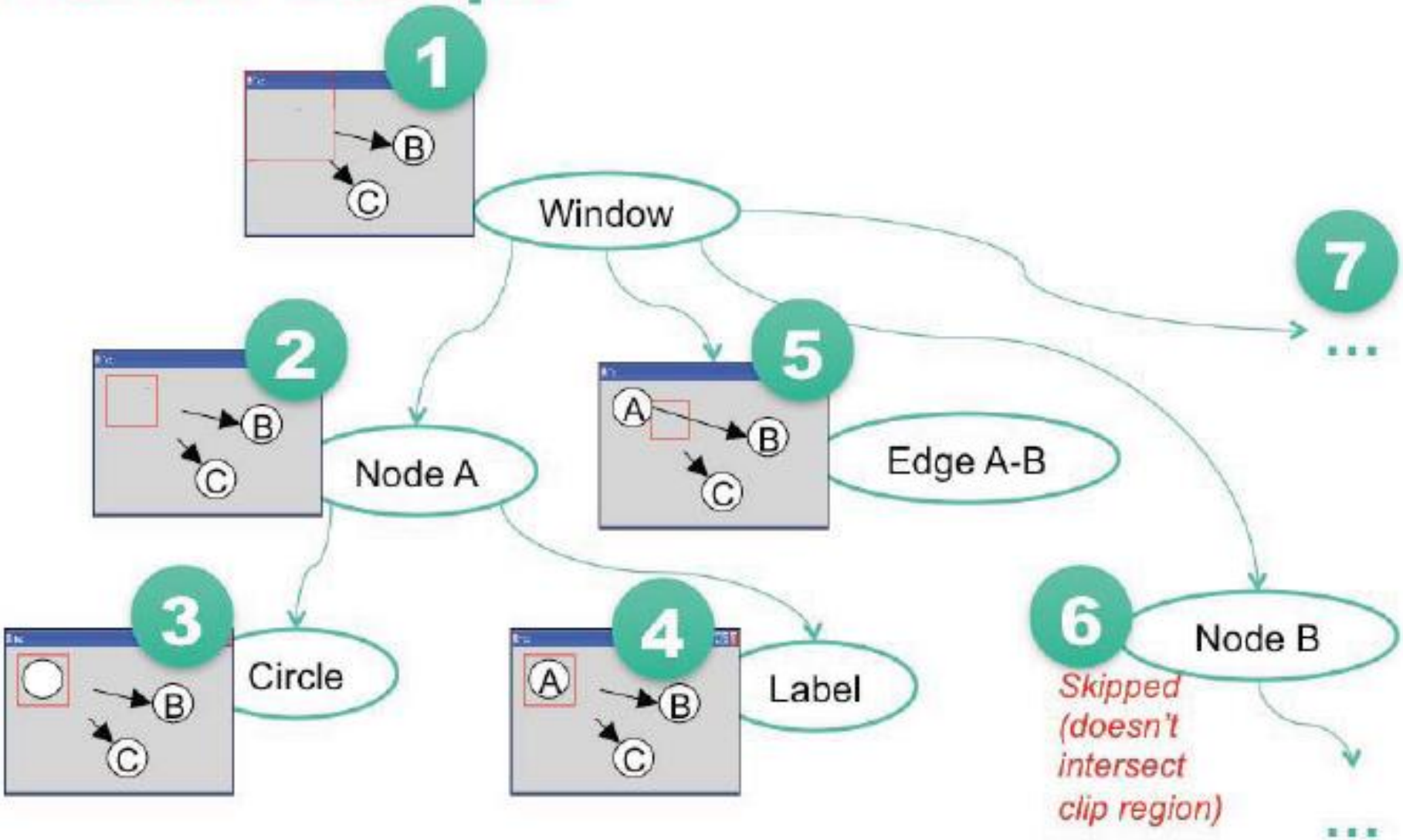


clipping region indicates the area of the screen that needs to be drawn

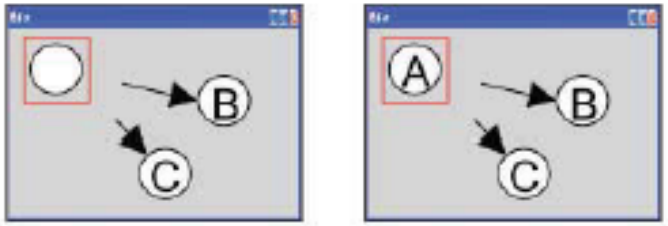
Non-rectangular bounding box



Redraw Example

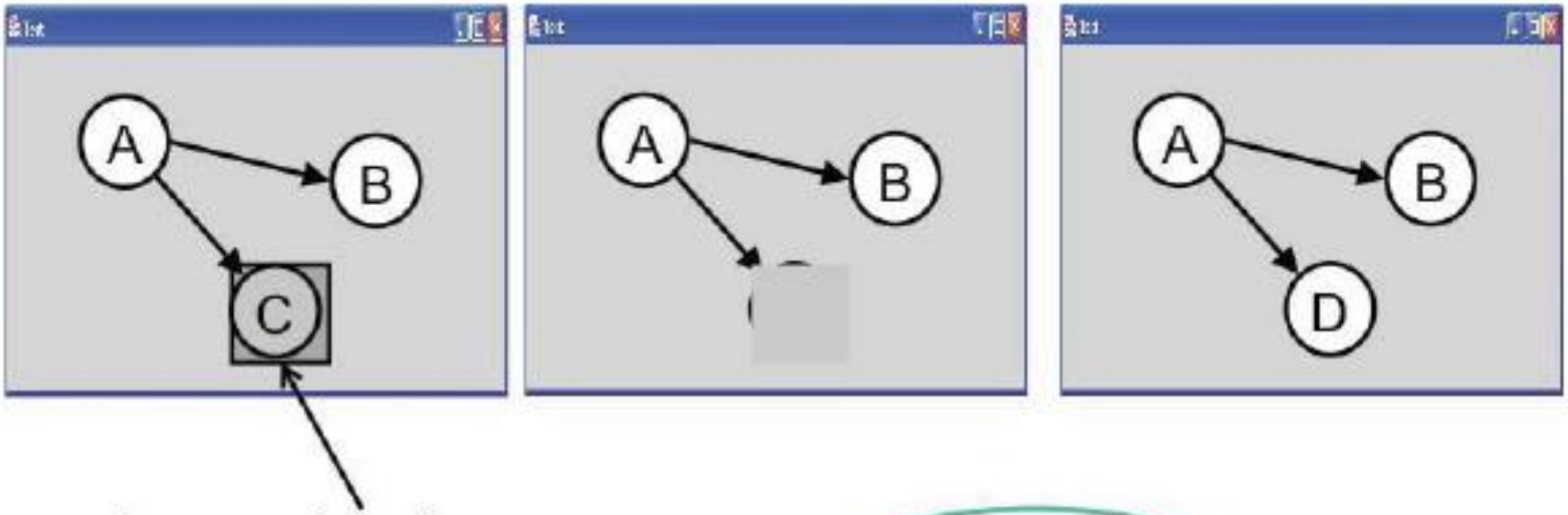


Z Order

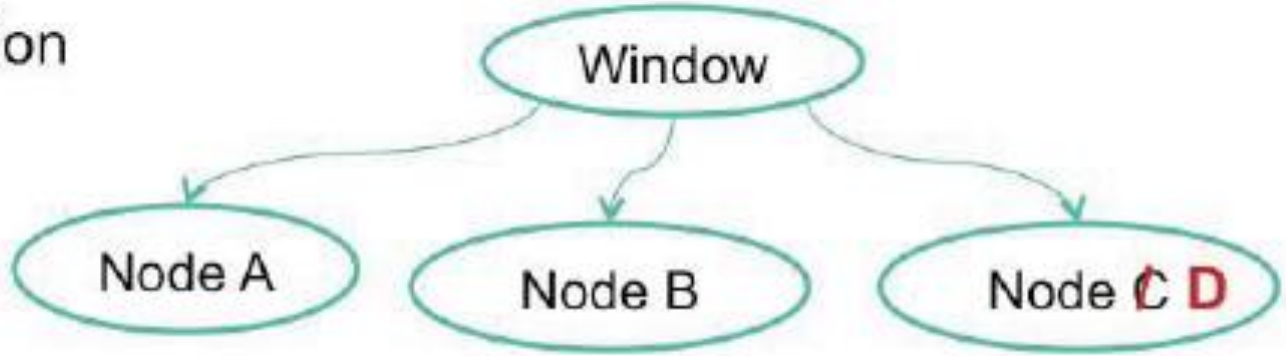


- 2D GUIs are really "2 ½ D"
  - Drawing order produces layers
  - Not a true z coordinate for each object, but merely an **ordering** in the z dimension
- View tree and redraw algorithm dictate z order
  - Parents are drawn first, underneath children
  - Older siblings are drawn under younger ones
    - Flex, HTML, most GUI toolkits and drawing programs behave this way
    - Java Swing is backwards: last component added (highest index) is drawn first

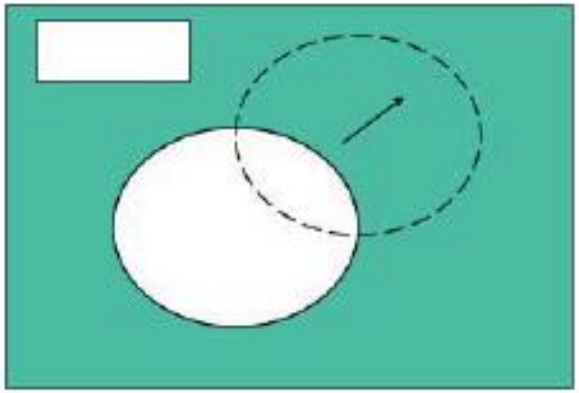
Damage and Automatic Redraw



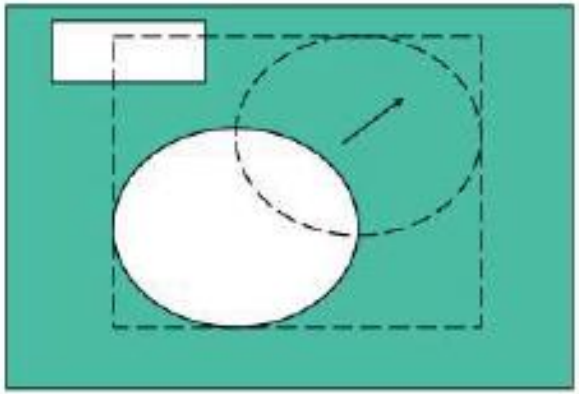
damaged region



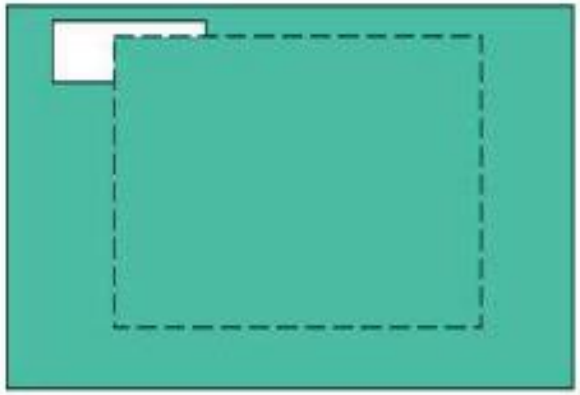
Naïve Redraw Causes Flashing Effects



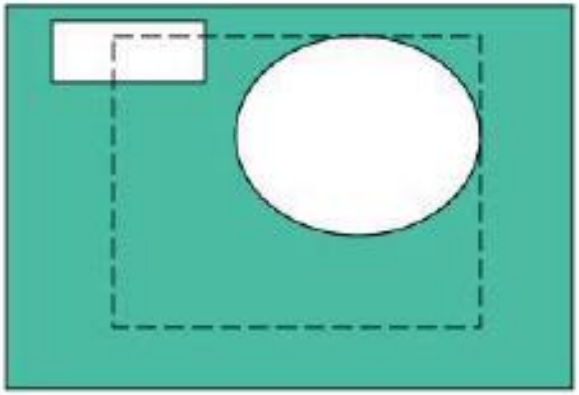
Object moves



Determine damaged region



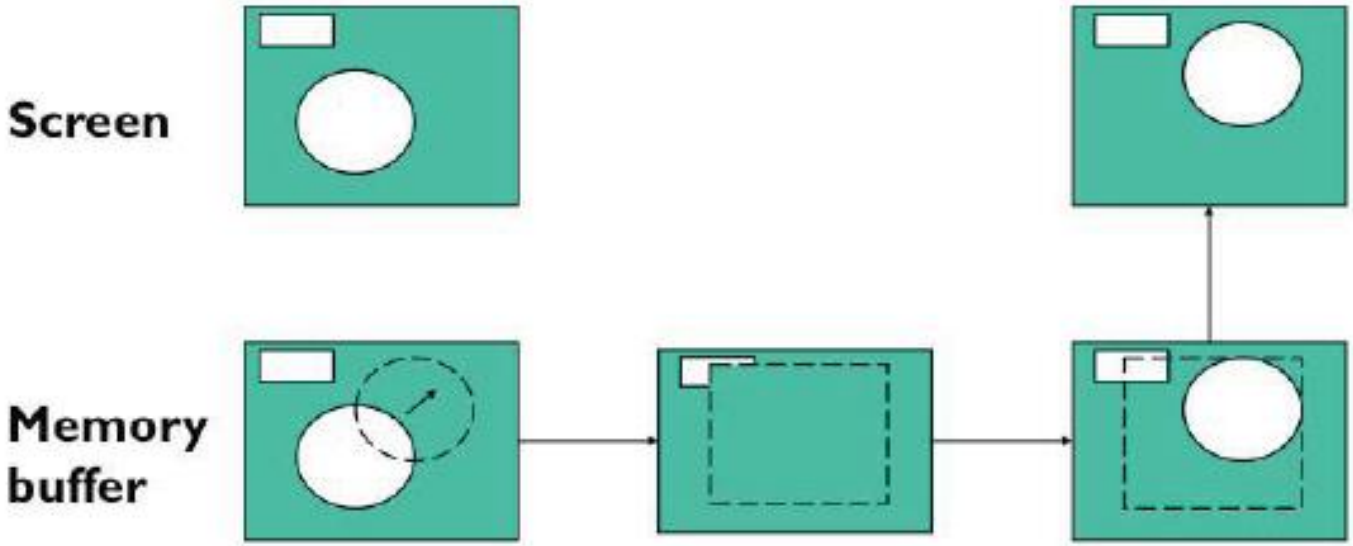
Redraw parent (children blink out!)



Redraw children

Double-Buffering

- Double-buffering solves the flashing problem



pixel copy is faster than redrawing the view hierarchy, so there's less chance that a screen refresh will catch it half-done
unmoving objects that happen to be caught, as innocent victims, in the damaged region are never erased from the screen, only from the memory buffer

Going From Objects to Strokes

- Drawing method approach
  - e.g. Swing paint() method
  - Drawing method is called directly during redraw; override it to change how component draws itself
- Retained graphics approach
  - e.g. Adobe Flex
  - Stroke calls are recorded and played back at redraw time
- Differences
  - Retained graphics is less error prone
  - Drawing method gives more control and performance

Stroke Model

- Drawing surface
  - Also called drawable (X Windows), GDI (MS Win)
  - Screen, memory buffer, print driver, file, remote screen
- Graphics context
  - Encapsulates drawing parameters so they don't have to be passed with each call to a drawing primitive
  - Font, color, line width, fill pattern, etc.
- Coordinate system
  - Origin, scale, rotation
- Clipping region
- Drawing primitives
  - Line, circle, ellipse, arc, rectangle, text, polyline, shapes

HTML5 Canvas in One Slide

HTML element

```
<canvas width=1000
   height=1000></canvas>
```

graphics context

```
var ctx = canvas.getContext
   ("2d")
```

coordinate system

```
ctx.translate()
ctx.rotate()
ctx.scale()
```

color, font, line style, etc.

```
ctx.strokeStyle = "rgb
   (0.5,0.5,0.5)"
ctx.fillStyle = …
ctx.font = "bold 12pt sans-
   serif"
ctx.lineWidth = 2.5
```

drawing primitives

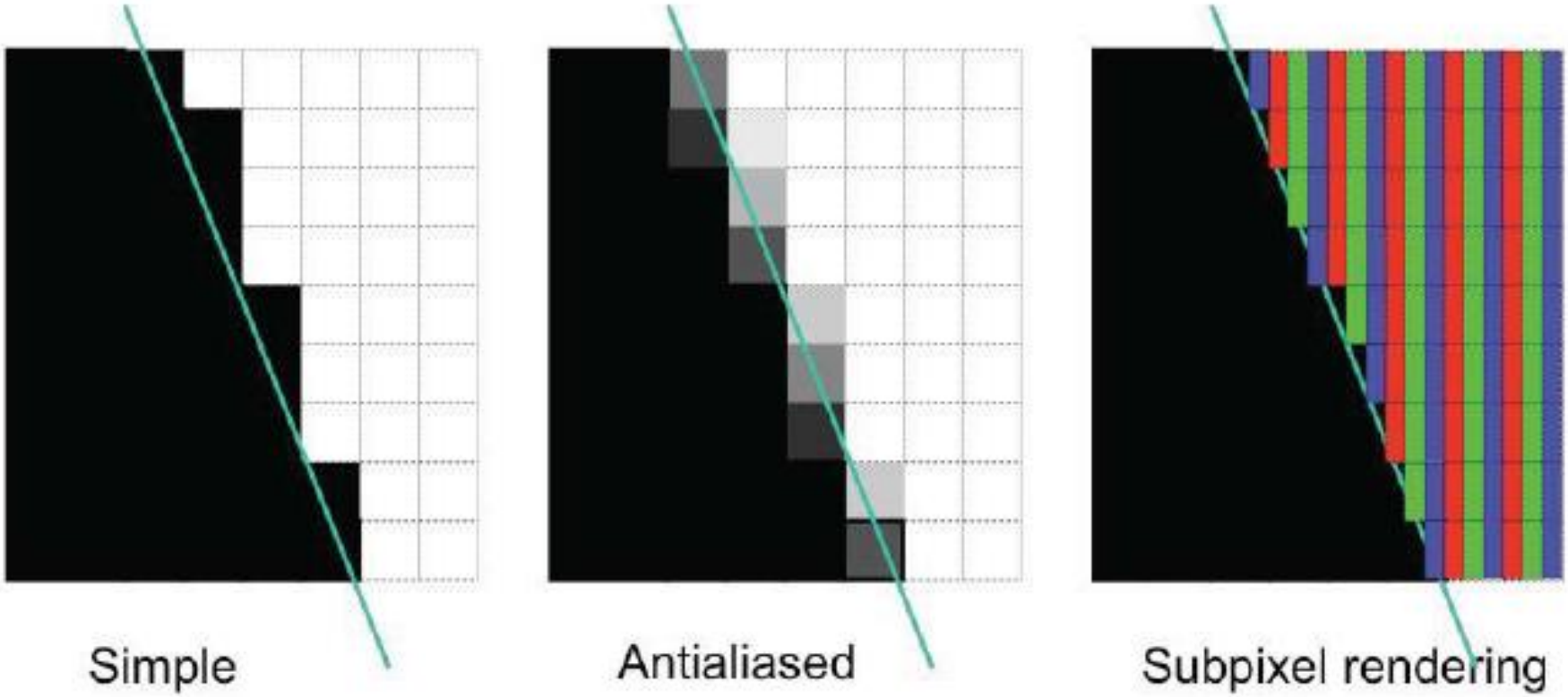
```
ctx.beginPath();
ctx.moveTo(0,0)
ctx.lineTo(500,500)
ctx.stroke()

ctx.beginPath()
ctx.arc(500,500,100,0,
   2*Math.PI,false)
ctx.fill()
```

clipping

```
ctx.beginPath()
ctx.rect(0, 0, 200, 300)
ctx.clip()
```

Antialiasing and Subpixel Rendering



Pixel Approach

- Pixel approachis a rectangular array of pixels
  - Each pixel is a vector (e.g., red, green, blue components), so pixel array is really 3 dimensional
- Bits per pixel (bpp)
  - 1 bpp: black/white, or bit mask
  - 4-8 bpp: each pixel is an index into a color palette
  - 24 bpp: 8 bits for each color
  - 32 bpp: 8 bits for each color + alpha channel
- Color components (e.g. RGB) are also called channels or bands
- Pixel model can be arranged in many ways
  - Packed into words (RGBR GBRG ...) or loosely (RGB- RGB- ...)
  - Separate planes (RRR...GGG...BBB...) vs. interleaved (RGB RGB RGB...)
  - Scanned from top to bottom vs. bottom to top

Transparency

- **Alpha** is a pixel's transparency
  - from 0.0 (transparent) to 1.0 (opaque)
  - so each pixel has red, green, blue, and alpha values
- Uses for alpha
  - Antialiasing
  - Nonrectangular images
  - Translucent components
  - Clipping regions with antialiased edges

Image File Formats

- GIF
  - 8 bpp, palette uses 24-bit colors
  - 1 color in the palette can be transparent (1-bit alpha channel)
  - lossless compression
  - suitable for screenshots, stroked graphics, icons
- JPEG
  - 24 bpp, no alpha
  - lossy compression: visible artifacts (dusty noise, moire patterns)
  - suitable for photographs
- PNG
  - lossless compression
  - 1, 2, 4, 8 bpp with palette
  - 24 or 48 bpp with true color
  - 32 or 64 bpp with true color and alpha channel
  - suitability same as GIF
  - better than GIF, but no animation

Hints for Debugging Output

- Something you're drawing isn't appearing on the screen. Why not?
  - Wrong place
  - Wrong size
  - Wrong color
  - Wrong z-order

Summary

- Component, stroke, pixel models
- Automatic redraw and double-buffering
- Image formats
- Declarative UI: markup languages and CSS